# AutoML에서 Autoresearch로

DOE는 Research Agent의 Harness가 될 수 있는가?

AutoML, Autoresearch, MLOps, DOE-guided Agent

# 1. 왜 지금 이 주제인가?

- Coding Agent는 `test`와 `CI` 위에서 빠르게 발전하고 있다.
- Research automation도 문헌, 코드, 실험, 보고서까지 넓어지고 있다.
- 이제 질문은 실행 가능성보다 `어떤 harness가 연구 실험을 규율하는가`이다.

| 빠르게 강해지는 것 | 아직 약한 것 |
|---|---|
| Coding Agent의 실제 배포 | research loop의 규율 |
| end-to-end automation demo | reproducibility / robustness |

## 2. AutoML이란 무엇인가?

- AutoML은 모델 개발의 탐색 일부를 자동화한다.
- 대표 대상은 `model selection`, `hyperparameter tuning`, `pipeline search`다.
- 핵심은 비교적 주어진 `search space` 안 최적화다.

```
Data → Search over models/pipelines → Evaluation → Best configuration
```

# 3. Autoresearch란 무엇인가?

- Autoresearch는 연구 workflow 일부를 agent가 직접 수행하는 형태다.
- 대상은 `config`가 아니라 `가설`, `code`, `pipeline`, `experiment plan`까지 넓어진다.
- 목표도 단일 최고 설정보다 더 나은 실험 program을 찾는 데 가깝다.

`Question → Read → Edit → Run → Analyze → Next experiment`

# 4. Autoresearch agent는 어떻게 일하는가?

| 단계 | agent가 하는 일 |
|---|---|
| context build | 논문, 코드, 이전 결과를 읽는다 |
| planning | 다음 실험과 baseline을 고른다 |
| execution | 코드를 바꾸고 실행한다 |
| interpretation | 결과를 해석하고 다음 실험으로 넘긴다 |

- 즉 Autoresearch는 한 번의 search보다 `iterative research loop`에 가깝다.

# 5. AutoML vs. Autoresearch

| 항목 | AutoML | Autoresearch |
|---|---|---|
| 탐색 대상 | config, architecture | hypothesis, code, module, pipeline, experiment program |
| 평가 | explicit objective | objective + reasoning + iteration |
| 주요 위험 | 비효율적 탐색 | metric hacking, incoherent search |
| 필요한 인프라 | experiment infra | experiment + memory + harness |

# 6. 현재 사용례

| 연구 작업 | 이미 자동화되는 형태 |
|---|---|
| 지식 수집 | 논문 검색, 요약, citation report |
| 아이디어 정리 | novelty check, proposal refinement |
| 실험 실행 | code edit, run, compare, keep-or-revert |
| 결과 산출 | analysis, draft, figure, reviewer response |

# 7. 발전 방향

- `topic → experiment → paper`를 한 loop로 묶는 시도
- Coding Agent를 실행층으로 쓰는 구조
- `benchmark`, `memory`, `reusable skills` 강화
- consumer GPU, WebGPU, swarm 등 infra 다변화

# 8. 자동화를 가능하게 하는 공통 기반: MLOps

- 반복 자동화에는 `tracking`, `orchestration`, `reproducibility`가 필요하다.
- 여기에 `artifact`, `monitoring`, `cost governance`가 붙어야 운영 가능성이 생긴다.
- 그래서 `MLOps`는 AutoML 뒤처리가 아니라 둘의 공통 기반이다.

| 공통 인프라 | AutoML에서의 역할 | Autoresearch에서의 역할 |
|---|---|---|
| tracking | 후보 비교 | 실험 history 축적 |
| orchestration | search job 실행 | multi-step agent loop 실행 |
| reproducibility | 재실행 / 검증 | hypothesis와 edit 검증 |

# Prelim 정리

배경은 여기까지다.

이제 질문은

`Research Agent에 어떤 harness가 필요한가?`

# 9. Harness gap

| Coding Agent 쪽에서 이미 흔한 것 | Research Agent 쪽에서 아직 약한 것 |
|---|---|
| `TDD` | hypothesis discipline |
| `CI` | factor definition |
| regression test | robustness / replication |
| deploy guardrail | experiment promotion rule |

- 그래서 높은 점수만으로는 좋은 연구 탐색인지 말하기 어렵다.

# 10. DOE를 Harness 후보로 보기

| DOE primitive | Agent loop에서의 역할 |
|---|---|
| screening | 중요한 factor를 초기에 좁힘 |
| factorial thinking | interaction을 구조적으로 확인 |
| sequential refinement | 유망 구간을 단계적으로 세밀화 |
| robustness / confirmation | 우연한 win과 재현 가능한 win을 분리 |

- 즉 DOE는 실험 순서와 비교 규칙을 준다.

# 11. Agent loop에 무엇이 달라지는가?

| 단계 | Vanilla Agent | DOE-guided Agent |
|---|---|---|
| 초기 | idea → edit → run | factor 정의 → screening |
| 중반 | gain 추적 | interaction check |
| 후반 | best-so-far 갱신 | refinement |
| 승격 | high score 채택 | robustness 후 promote |

# 12. 실험 설정

| Variant | 설명 | 비교 목적 |
|---|---|---|
| Vanilla Agent | unconstrained iterative experiments | 기본 baseline |
| DOE Agent | DOE 기반 planning과 staging | harness 효과 측정 |
| DOE + X Agent | DOE + 추가 전략 모듈 | harness와 추가 전략의 결합 효과 |

- 비교 축: `score`, `cost`, `robustness`, `portfolio behavior`

# 13. 결과 요약

`TBD after experiments`

| Variant | Best score | Avg improvement | Runs | Cost | Robustness pass |
|---|---|---|---|---|---|
| Vanilla | TBD | TBD | TBD | TBD | TBD |
| DOE | TBD | TBD | TBD | TBD | TBD |
| DOE + X | TBD | TBD | TBD | TBD | TBD |

# 14. 성능 향상 추이

- `TBD after experiments`
- `experiment budget` 또는 `run count` 대비 `best-so-far metric`
- 수렴 속도, plateau 형태, sample efficiency 비교

`x-axis = budget / run count`

`y-axis = best-so-far metric`

# 15. 실험 포트폴리오 또는 실패 구조

- `TBD after experiments`
- `Tic / Tac / To` 비율
- 또는 실패 분류
- 또는 `information gain vs. performance gain`

# 16. 왜 DOE-guided Agent는 다를 수 있는가?

- `search space`가 더 구조화된다.
- 중복되거나 정보량이 낮은 실험이 줄어든다.
- 상호작용을 우연히가 아니라 의도적으로 본다.
- robustness 단계가 분리된다.

# 17. 대학원 연구 루프는 더 넓다

- 문헌 조사
- 가설 수립
- 실험 설계
- 구현 및 실행
- 분석과 토론
- 다음 실험 선택

# 18. DOE가 주로 커버하는 구간

| 구간 | DOE의 역할 |
|---|---|
| 강함 | 실험 설계, 비교, refinement, 다음 실험 선택 |
| 부분적 | 구현 및 실행, 결과 분석 |
| 약함 | 문헌 조사, 가설 생성 |

- 즉 DOE는 `research loop` 전체보다 experimentation layer를 강하게 규율한다.

# 19. 한계

- `literature review`나 `hypothesis generation` 자체를 대체하진 못한다.
- open-ended research space는 factorization이 어렵다.
- large-scale training에서는 replication 비용이 크다.
- evaluation harness가 약하면 DOE도 흔들린다.

# 20. 결론

- AutoML에서 Autoresearch로 갈수록 `search space` 는 넓어진다.
- search space가 넓어질수록 더 강한 `harness` 가 필요하다.
- DOE는 Research Agent의 experimentation harness로 매우 유력한 후보다.

> 다음 단계는 더 좋은 agent 자체보다 더 좋은 harness이다.

# 21. References

| 구분 | 예시 |
|---|---|
| curated landscape | `alvinreal/awesome-autoresearch`, `handsome-rich/Awesome-Auto-Research-Tools` |
| end-to-end systems | `karpathy/autoresearch`, `microsoft/RD-Agent`, `SakanaAI/AI-Scientist` |
| deep research | `assafelovic/gpt-researcher`, `Open Deep Research`, `PaperQA2` |
| experiment/code agents | `OpenHands`, `Aider`, `SWE-agent`, `AIDE` |
| evaluation | `openai/mle-bench`, `snap-stanford/MLAgentBench`, `chchenhui/mlrbench` |